Projeto Final: Sistema de Regressão Espectral com Bagging e Subagging

Reengenharia e Implementação em Streamlit/Python

Wagner Oliveira de Araujo

IFG — 2026

Sumário

1 Levantamento **3**
1.1 Classificação por Prioridade 4

2 Projeto Lógico **6**

3 Projeto Físico **7**

4 Implementação **7**
4.1 Destaques da Implementação (`app.py`) 8

1 Levantamento

O levantamento de requisitos visou catalogar, classificar e priorizar as funcionalidades do sistema legado (Vaadin/Java) para orientar a reengenharia em Python/Streamlit. A análise partiu do mapeamento exaustivo da base de código existente, garantindo rastreabilidade e aderência às necessidades de pesquisadores e alunos de quimiometria.

Conforme detalhado no Quadro 1, o inventário categoriza as funcionalidades em grupos lógicos (Cálculos, Configuração, Visualização, Organização e Exportação), evidenciando que o sistema original focava estritamente na execução algorítmica, sem mecanismos nativos de exportação ou persistência de resultados.

Quadro 1 – Inventário das funções do sistema

Categoria	**Funcionalidade**	**Descrição Técnica**
Cálculos	RLM Simples (Baseline)	Regressão Linear Múltipla padrão sem técnicas de ensemble
Cálculos	RLM + Bagging	Ensemble com amostragem *com* reposição (bootstrap)
Cálculos	RLM + Subagging + CV	Ensemble com amostragem *sem* reposição + otimização por validação cruzada 5-Fold
Configuração	Parâmetro m (Bags)	Define número de amostras/ensembles para o método Bagging
Configuração	Parâmetro k (Subagging)	Define fração ou quantidade para amostragem no Subagging
Visualização	Métricas de Desempenho	Exibição de R^2, RMSE, MAE calculados no conjunto de teste
Visualização	Gráficos Comparativos	*Scatter plots* (previsto $\times$ real), resíduos e comparação entre modelos
Organização	Abas de Navegação	Separação lógica: "Arquivos", "Configuração", "Resultados", "Gráficos"
Exportação	*Não identificada*	**Obs.:** sistema não apresenta botões de download de resultados

Fonte: Wagner Oliveira de Araujo

Para que os algoritmos sejam executados corretamente, é necessário fornecer matrizes espectrais e vetores de referência em formato texto. Os arquivos e parâmetros exigidos estão listados no Quadro 2, que especifica a estrutura dimensional esperada para calibração e teste, além dos limites válidos para os hiperparâmetros m e k.

Após o processamento dos dados de entrada, o sistema gera indicadores quantitativos e representações visuais para suporte à decisão técnica. As saídas geradas pelo sistema são apresentadas no Quadro 3, descrevendo os formatos de exibição adotados para métricas, gráficos comparativos, resíduos e feedbacks de interface.

Quadro 2 – Entradas necessárias para execução do sistema

Tipo	Nome/Parâmetro	Descrição
Arquivo	`Xcal.txt`	Matriz de espectros de calibração (variáveis independentes)
Arquivo	`Ycal.txt`	Vetor de valores de referência para calibração
Arquivo	`Xteste.txt`	Matriz de espectros para predição/teste
Arquivo	`Yteste.txt`	Vetor de valores reais para validação do conjunto de teste
Parâmetro	m (nº de bags)	Inteiro ≥ 1. Usado apenas nos métodos de ensemble
Parâmetro	k (Subagging)	Decimal $(0, 1]$. Usado apenas no método Subagging
Config.	Validação Cruzada	Fixo: 5-Fold. Parâmetro interno de otimização do modelo

Fonte: Wagner Oliveira de Araujo

Quadro 3 – Saídas esperadas do sistema

Tipo	Conteúdo	Formato de Exibição
Numérico	R^2, RMSE, MAE (teste)	Tabela ou cards destacados
Gráfico	Previsto vs. Real	Scatterplot com reta de ajuste
Gráfico	Resíduos da Predição	Gráfico de dispersão ou histograma
Comparativo	Desempenho entre modelos	Tabela comparativa ou gráfico de barras
Feedback	Mensagens de erro/validação	Alertas em destaque na interface
Status	Indicador de processamento	Spinner ou barra de progresso

Fonte: Wagner Oliveira de Araujo

1.1 Classificação por Prioridade

A priorização foi realizada por meio da técnica MoSCoW[1], separando as funcionalidades em essenciais, desejáveis e extras. Conforme o Quadro 4, os requisitos essenciais (Must Have) garantem o funcionamento mínimo do sistema, abrangendo desde a validação dimensional até a exibição clara dos resultados para tomada de decisão.

Quadro 4 – Requisitos essenciais do sistema (Must Have)

ID	Requisito	Justificativa
E01	Upload e validação dos 4 arquivos	Dados são pré-requisito para qualquer cálculo estatístico
E02	Execução da RLM Simples (baseline)	Referência obrigatória para comparação de desempenho
E03	Cálculo de métricas básicas (R^2, RMSE, MAE)	Mínimo necessário para avaliação quantitativa da qualidade
E04	Validação dimensional (X vs Y)	Previne erros de execução e resultados inválidos
E05	Exibição clara dos resultados	Permite decisão técnica fundamentada em evidências

Fonte: Wagner Oliveira de Araujo

Os requisitos desejáveis não bloqueiam a execução, mas agregam valor significativo à experiência do usuário e à reprodutibilidade da pesquisa. O Quadro 5 lista os itens classifi-

[1]MoSCoW: acrônimo para *Must Have* (obrigatório), *Should Have* (importante), *Could Have* (desejável) e *Won't Have* (fora do escopo atual). Método amplamente adotado em engenharia de software e gestão ágil de projetos para priorização de requisitos.

cados como Should Have, focados em usabilidade avançada, exportação de dados e suporte contextual por meio de tooltips e histórico de sessão.

Quadro 5 – Requisitos desejáveis do sistema (Should Have)

ID	Requisito	Benefício
D01	Gráficos interativos (zoom, tooltip)	Facilita exploração visual e apresentação de resultados
D02	Exportação de resultados (CSV, PDF, PNG)	Permite documentação e reprodutibilidade da análise
D03	Tooltips explicativos para m, k	Reduz curva de aprendizado para usuários não especialistas
D04	Pré-processamento básico (normalização)	Melhora robustez com dados espectrais reais
D05	Histórico de execuções na sessão	Permite comparar configurações sem reenviar arquivos

Fonte: Wagner Oliveira de Araujo

Para fases futuras de evolução do sistema, foram mapeados requisitos que ampliam a escalabilidade e a integração com ecossistemas de dados. Conforme o Quadro 6, os itens Could Have incluem autenticação, suporte a múltiplos formatos e pipelines automatizadas, alinhando a ferramenta a demandas corporativas e acadêmicas avançadas.

Quadro 6 – Requisitos extras para fases futuras (Could Have)

ID	Requisito	Cenário de Uso
X01	Autenticação e projetos privados	Ambientes corporativos ou pesquisa com dados sensíveis
X02	Integração com APIs externas	Pipelines automatizadas de quimiometria
X03	Suporte a formatos (.csv, .xlsx, .mat)	Flexibilidade para ingestão de diferentes fontes
X04	Otimização automática de hiperparâmetros	Reduz necessidade de conhecimento técnico avançado
X05	Dashboard comparativo multi-rodada	Análise estatística de estabilidade dos ensembles

Fonte: Wagner Oliveira de Araujo

A robustez do sistema depende também do tratamento proativo de falhas e da mitigação de pontos críticos de usabilidade. A matriz de exceções, apresentada no Quadro 7, estabelece comportamentos esperados para entradas inválidas, timeouts e navegação inconsistente, garantindo que o usuário seja orientado a corrigir o fluxo sem perda de contexto.

Além dos requisitos funcionais, o sistema deve atender a restrições técnicas que garantem qualidade, segurança e escalabilidade. O Quadro 8 especifica os limites aceitáveis para tempo de resposta, critérios de acessibilidade WCAG 2.1 AA, isolamento de sessão e arquitetura de deploy, servindo como base para os testes de aceite e homologação.

Quadro 7 – Tratamento de exceções e usabilidade

Cenário	**Comportamento do Sistema**	**Mitigação Proposta**
Arquivo inválido	Alerta vermelho + destaque no campo	Validação dimensional *fail-fast* antes do processamento
Parâmetros fora do intervalo	Validação em tempo real + mensagem	Sliders com valores recomendados e limites explícitos
Timeout de processamento	Mensagem orientativa de otimização	Barra de progresso com etapas descritivas
Navegação sem executar	Persistência de uploads na sessão	Estado mantido em `st.session_state`
Resultados não exportáveis	Impede documentação externa	Botão "Baixar Relatório" com métricas + gráficos

Fonte: Wagner Oliveira de Araujo

Quadro 8 – Especificação de requisitos não funcionais

Categoria	**Especificação**	**Critério de Aceite**
Performance	Tempo de cálculo (RLM/Ensemble)	RLM $< 10\,\text{s}$; Ensemble $< 2\,\text{min}$ (CV 5-Fold)
Performance	Renderização de gráficos	$< 3\,\text{s}$ após conclusão do cálculo
Usabilidade	Acessibilidade (WCAG 2.1 AA)[a]	Contraste, labels, navegação por teclado
Usabilidade	Clareza textual e recuperação de erro	Mensagens orientam correção, não apenas apontam falha
Segurança	Isolamento de sessão e sanitização	Dados não vazam; validação de extensão/conteúdo
Escalabilidade	Crescimento de usuários/dados	Container com load balancer; processamento em batch

Fonte: Wagner Oliveira de Araujo

[a] WCAG (*Web Content Accessibility Guidelines*) 2.1, nível AA: padrão internacional do W3C que define critérios de acessibilidade digital para pessoas com deficiência. O nível AA é o exigido pela maioria das legislações, incluindo a Lei Brasileira de Inclusão (LBI). Disponível em: https://www.w3.org/TR/WCAG21/.

2 Projeto Lógico

A reengenharia adotou o modelo clássico de desenvolvimento, promovendo a separação de responsabilidades e a eliminação de acoplamentos presentes no código legado. O diagnóstico técnico e as soluções propostas estão sintetizados no Quadro 9, que contrasta a arquitetura monolítica original com a nova estrutura em camadas, destacando a introdução de validação explícita, vetorização NumPy e tipagem estática.

O fluxo lógico segue: `Upload` $\rightarrow$ `Validação dimensional` $\rightarrow$ `Configuração de` m `e` k $\rightarrow$ `Execução com CV 5-Fold` $\rightarrow$ `Exibição de resultados e gráficos`. A camada de negócios foi isolada para garantir que a matemática da regressão (pseudoinversa, bootstrap, subamostragem) seja testável independentemente da interface, reduzindo a probabilidade de regressões em futuras manutenções.

Quadro 9 – Diagnóstico do código legado e arquitetura proposta

Aspecto	**Estado Atual (app.py)**	**Solução na Reengenharia**
Arquitetura	Monolítica (UI + lógica + parsing)	Camadas separadas: Apresentação, Negócio, Dados
Validação	`try/except` genérico	Função `validate_inputs()` com regras explícitas
Performance	Laços `for` em CV, sem cache	`@st.cache_data`, vetorização NumPy, seed exposto
Tipagem/Doc	Ausente em funções críticas	Type hints + docstrings padrão NumPy
Reprodutibilidade	`seed=42` hardcoded	Parâmetro configurável + registro em logs

Fonte: Wagner Oliveira de Araujo

3 Projeto Físico

A organização física do projeto, conforme o Quadro 10, segue o princípio de responsabilidade única. Cada módulo reside em diretórios especializados, permitindo que a camada de apresentação (`app.py`) se comunique apenas com módulos de negócio (`src/`), enquanto os testes unitários (`tests/`) validam isoladamente as funções de parsing, métricas e otimização.

Quadro 10 – Estrutura modular e responsabilidades

Módulo/Arquivo	**Função Específica**
`app.py`	Camada de apresentação (apenas UI + roteamento de abas)
`src/data_loader.py`	Parsing de `.txt`, validação dimensional, dataclasses
`src/models.py`	Implementação de RLM, Bagging e Subagging (vetorizado)
`src/metrics.py`	Cálculo de RMSE, MAE e R^2 com tipos explícitos
`src/optimizers.py`	Validação cruzada 5-Fold e busca de hiperparâmetros
`tests/`	Suíte de testes unitários (`pytest`) para cada módulo

Fonte: Wagner Oliveira de Araujo

O ambiente de execução utiliza Python 3.10+, Streamlit 1.30+, NumPy/Pandas para cálculo, Plotly para visualização interativa e `pytest` para validação automatizada. O cache de sessão e a alocação dinâmica de memória previnem `MemoryError` em datasets espectrais grandes, enquanto a estrutura modular facilita a substituição de componentes sem impacto na interface.

4 Implementação

A implementação materializa o projeto lógico e físico em código Python, mantendo fidelidade matemática ao original e ampliando usabilidade e reprodutibilidade. Para garantir a qualidade do código, o Quadro 11 define a estratégia de verificação, estabelecendo metas de cobertura por tipo de teste e critérios de aceite baseados em desempenho e experiência do usuário (SUS ≥ 75).

Quadro 11 – Estratégia de testes e critérios de aceite

Tipo de Teste	Escopo	Meta de Cobertura
Unitário	Funções isoladas (parse, metrics)	$\geq 80\%$ (`pytest-cov`)
Integração	Fluxo completo: Upload $\rightarrow$ CV $\rightarrow$ Gráficos	100% dos caminhos críticos
Performance	Tempo de resposta para datasets típicos	RLM < 10 s; Ensemble < 2 min
Aceite (UAT)[a]	Validação por pesquisadores/alunos	SUS[b] ≥ 75 (Bom/Excelente)

Fonte: Wagner Oliveira de Araujo

[a] UAT (*User Acceptance Testing*): Teste de Aceitação do Usuário — fase em que usuários reais validam se o sistema atende às suas necessidades antes da entrega final.

[b] SUS (*System Usability Scale*): questionário padronizado de 10 perguntas criado por John Brooke (1986) para medir a usabilidade percebida de um sistema. O *score* varia de 0 a 100, sendo: ≥ 85 Excelente, 75–84 Bom, 68–74 Aceitável e ≤ 67 Ruim.

4.1 Destaques da Implementação (`app.py`)

```python
def perform_bagging(Xcal, Ycal, Xteste, num_bags, with_replacement, fraction, seed=42):
    """RLM + Bagging / Subagging com vetorização e seed configurável."""
    n = Xcal.shape[0]
    sample_size = n if with_replacement else max(1, int(n * fraction))
    rng = np.random.default_rng(seed)
    Ypred = np.zeros(Xteste.shape[0])
    for _ in range(num_bags):
        idx = rng.integers(0, n, size=sample_size) if with_replacement else \
            rng.choice(n, size=sample_size, replace=False)
        beta = np.linalg.pinv(Xcal[idx]) @ Ycal[idx]
        Ypred += (Xteste @ beta) / num_bags
    return Ypred
```

A rastreabilidade completa é verificada por meio da matriz apresentada no Quadro 12, que vincula cada ID de requisito às fases clássicas de engenharia de software e aos artefatos de validação correspondentes. Essa matriz assegura que nenhuma funcionalidade levantada seja perdida durante a transição de requisitos para código, permitindo auditoria técnica e controle de escopo em todo o ciclo de desenvolvimento.

Quadro 12 – Matriz de rastreabilidade de requisitos (*Requirements Traceability Matrix* - RTM)

ID Requisito	Fase Clássica	Artefato de Validação
RF01.1–01.5	1, 3, 5	`data_loader.py`, testes de upload
RF02.1–02.3	1, 2, 4	`optimizers.py`, logs de execução
RF03.1–03.3	2, 3, 4	`models.py`, testes unitários de métricas
RF04.1–04.3	2, 4, 5	`metrics.py`, UAT visual (Plotly)
RF05.1–05.3	1, 3, 5	UI/UX, feedback de erros, `st.progress`

Fonte: Wagner Oliveira de Araujo

A implementação resultante atende aos requisitos funcionais e não funcionais levantados, reduz a curva de aprendizado para usuários não especialistas e estabelece base técnica sólida para evoluções futuras (autenticação, pipelines automatizados, suporte a novos formatos). A integração explícita entre texto e quadros, conforme demonstrado ao longo deste documento, garante conformidade com as normas acadêmicas e rastreabilidade técnica completa.